Experiment No. 3

[i] binary :

```
#include <stdio.h>

int main() {
    int arr[100], n, i, key;
    int low = 0, high, mid, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter sorted elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    high = n - 1;

    while(low <= high) {
        mid = (low + high) / 2;

        if(arr[mid] == key) {
            printf("Element found at position %d\n", mid + 1);
            found = 1;
            break;
        }
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if(found == 0)
        printf("Element not found\n");

    return 0;
}
```

```
C:\Users\PRASANNA\Desktop\DS practicals\3rd practical>cd "c:\
s\3rd practical\"binary
Enter number of elements: 10
Enter sorted elements:
23
43
54
67
89
90
91
102
104
159
Enter element to search: 89
Element found at position 5

c:\Users\PRASANNA\Desktop\DS practicals\3rd practical>
```

[ii] Linear :

```
#include <stdio.h>

int main() {
    int arr[100], n, i, key, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++) {
        if(arr[i] == key) {
            printf("Element found at position %d\n",
i + 1);
            found = 1;
            break;
        }
    }

    if(found == 0)
        printf("Element not found\n");

    return 0;
}
```

```
c:\Users\PRASANNA\Desktop\DS practical
s\3rd practical\"linear
Enter number of elements: 10
Enter elements:
43
12
56
76
44
98
23
56
33
12
Enter element to search: 44
Element found at position 5
```

[iii] : Hashing :

```
#include<stdio.h>

#define SIZE 7

int hashTable[SIZE];

void initialize()
{
   for(int i = 0; i < SIZE; i++)
      hashTable[i] = -1;   // -1 means empty
}

void insert(int key)
{
   int index = key % SIZE;
   int start = index;

   while(hashTable[index] != -1 && hashTable[index] != -2)
   {
      index = (index + 1) % SIZE;

      if(index == start)
      {
         printf("Hash Table is Full\n");
         return;
      }
   }

   hashTable[index] = key;
   printf("Key %d inserted at index %d\n", key, index);
}

void search(int key)
{
   int index = key % SIZE;
   int start = index;

   while(hashTable[index] != -1)
   {
      if(hashTable[index] == key)
      {
         printf("Key %d found at index %d\n", key, index);
         return;
      }

      index = (index + 1) % SIZE;
```

```
        if(index == start)
            break;
    }

    printf("Key not found\n");
}

void deleteKey(int key)
{
    int index = key % SIZE;
    int start = index;

    while(hashTable[index] != -1)
    {
        if(hashTable[index] == key)
        {
            hashTable[index] = -2;   // -2 means deleted
            printf("Key %d deleted\n", key);
            return;
        }

        index = (index + 1) % SIZE;

        if(index == start)
            break;
    }

    printf("Key not found\n");
}

void display()
{
    printf("\nHash Table:\n");
    for(int i = 0; i < SIZE; i++)
    {
        printf("Index %d : ", i);

        if(hashTable[i] == -1)
            printf("Empty\n");
        else if(hashTable[i] == -2)
            printf("Deleted\n");
        else
            printf("%d\n", hashTable[i]);
    }
}

int main()
{
    int choice, key;
```

```
    initialize();

    while(1)
    {
        printf("\n1. Insert\n2. Search\n3. Delete\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insert(key);
                break;

            case 2:
                printf("Enter key to search: ");
                scanf("%d", &key);
                search(key);
                break;

            case 3:
                printf("Enter key to delete: ");
                scanf("%d", &key);
                deleteKey(key);
                break;

            case 4:
                display();
                break;

            case 5:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

```
C:\Users\PRASANNA\Desktop\DS pract
\3rd practical\"hashing

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key to insert: 322
Key 322 inserted at index 0

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key to insert: 54
Key 54 inserted at index 5

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key to insert: 34
Key 34 inserted at index 6

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key to insert: 765
Key 765 inserted at index 2

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key to insert: 787
Key 787 inserted at index 3
```

```
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4

Hash Table:
Index 0 : 322
Index 1 : Empty
Index 2 : 765
Index 3 : 787
Index 4 : Empty
Index 5 : 54
Index 6 : 34

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 3
Enter key to delete: 787
Key 787 deleted

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4

Hash Table:
Index 0 : 322
Index 1 : Empty
Index 2 : 765
Index 3 : Deleted
Index 4 : Empty
Index 5 : 54
Index 6 : 34

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 5

c:\Users\PRASANNA\Desktop\DS pract
```